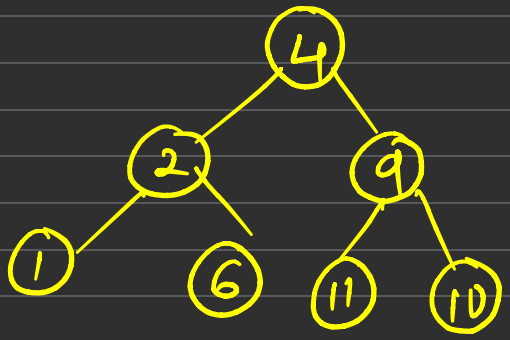
Your path is harder, maybe your calling is higher

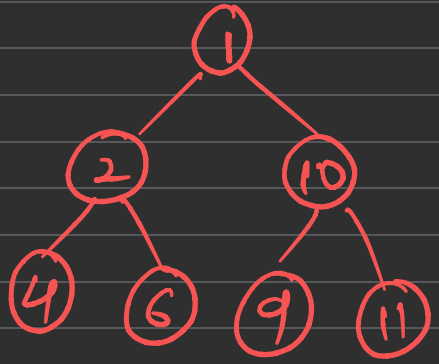
Tree

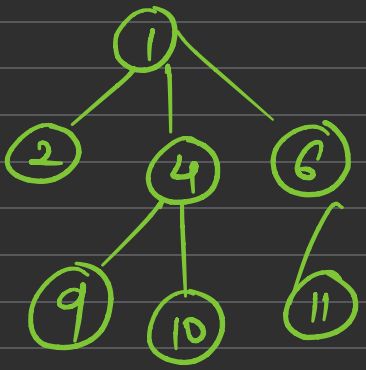
- Non-linear Data structure
- Parent-child Relation
- Nodes are connected by Edges

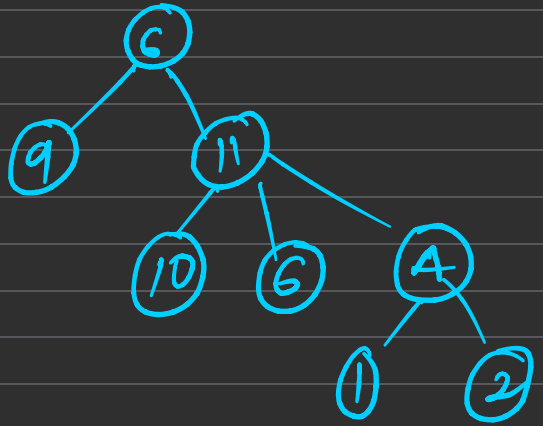
- Every element is represented in form of Nodes.
- Line which joins two Nodes → Edge

1 2 4 6 9 10 11









Terminologies of Tree

Parent Node: The node which is an immediate predecessor of a node

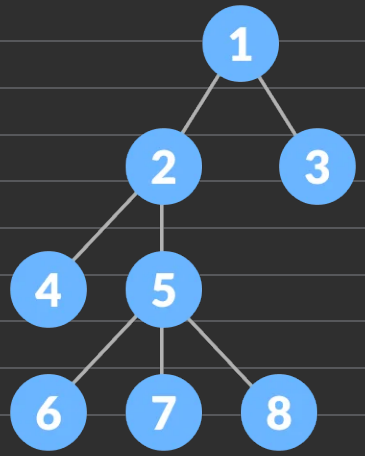
Child Node: The node which is the immediate successor of a node

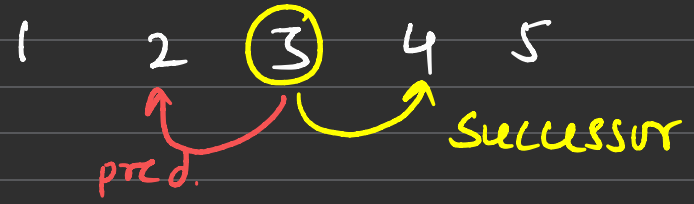
Root Node: The node which does not have any Parent.

Leaf Node: The node which does not have any children.

Sibling: Children of the same parent node are called siblings.

Height of Tree: Longest path from root node to deepest leaf node.





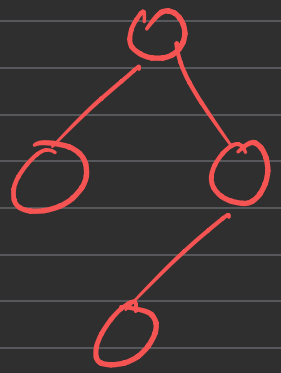
Types of Tree (BT)

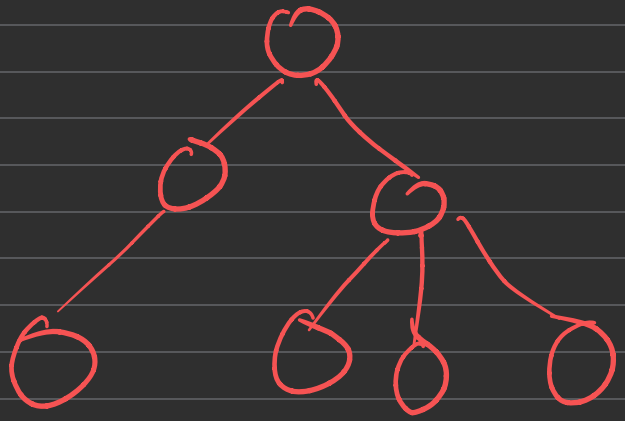
Binary Tree: In Binary Tree, each node can have almost 2 children

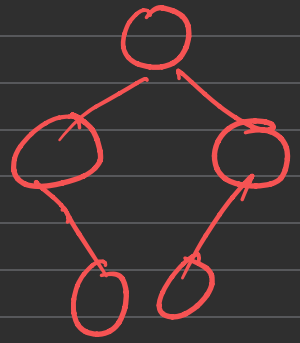
Complete Binary Tree: It is a binary tree, in which you can go to next level only if, one level is filled from left to right.

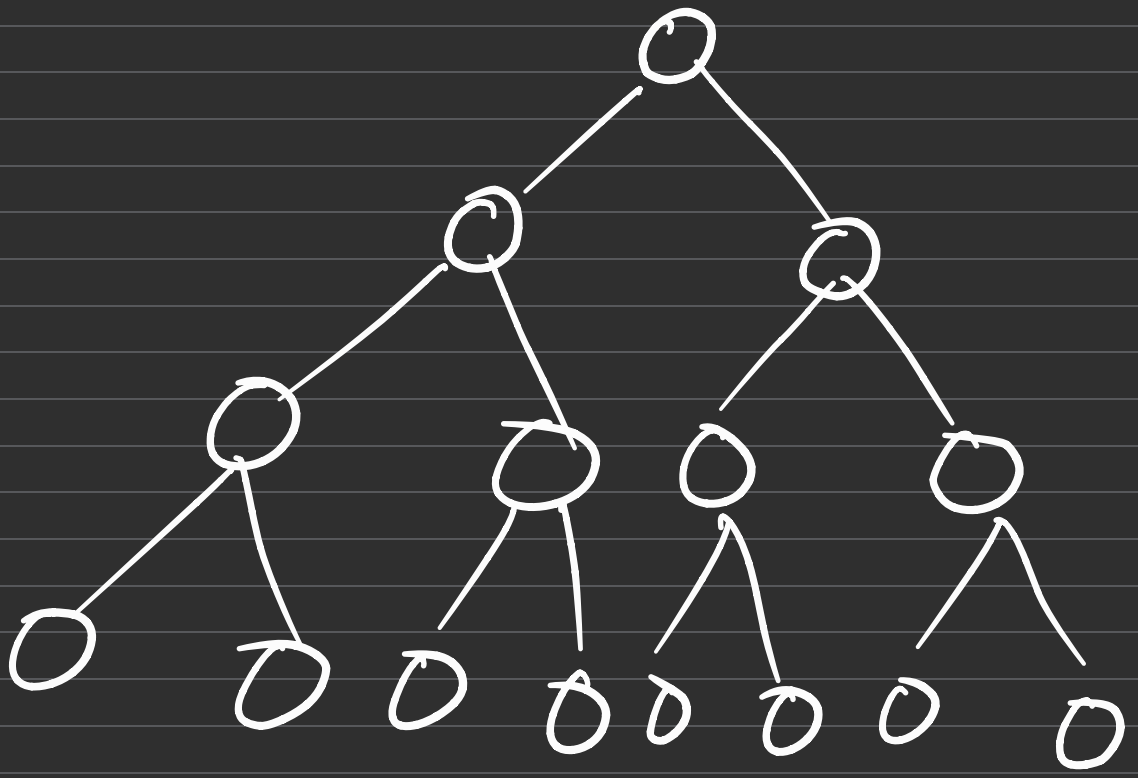
Full Binary Tree: It is a binary tree, in which a node can have either 2 children or no child.

- Binary Tree (0, 1, 2 children)









Level | No. of Nodes

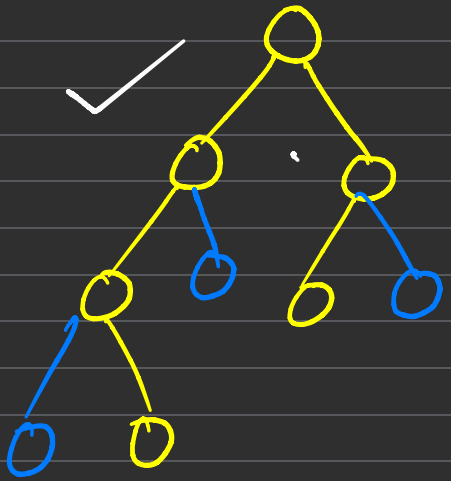
0 | 2^0

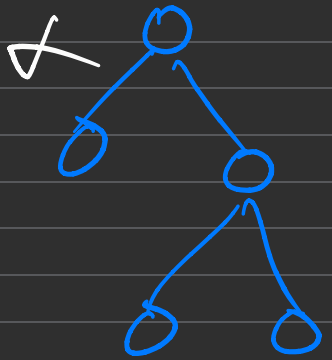
1 | 2^1

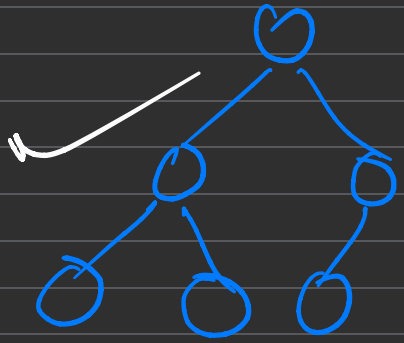
2 | 2^2

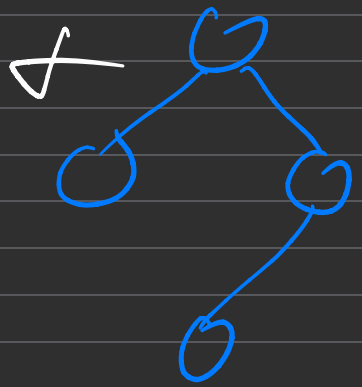
3 | 2^3

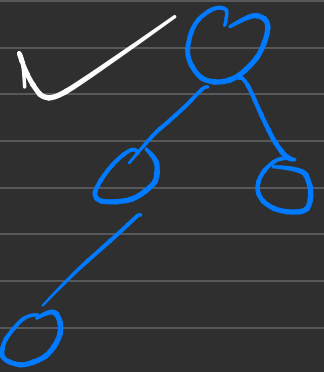
CBT



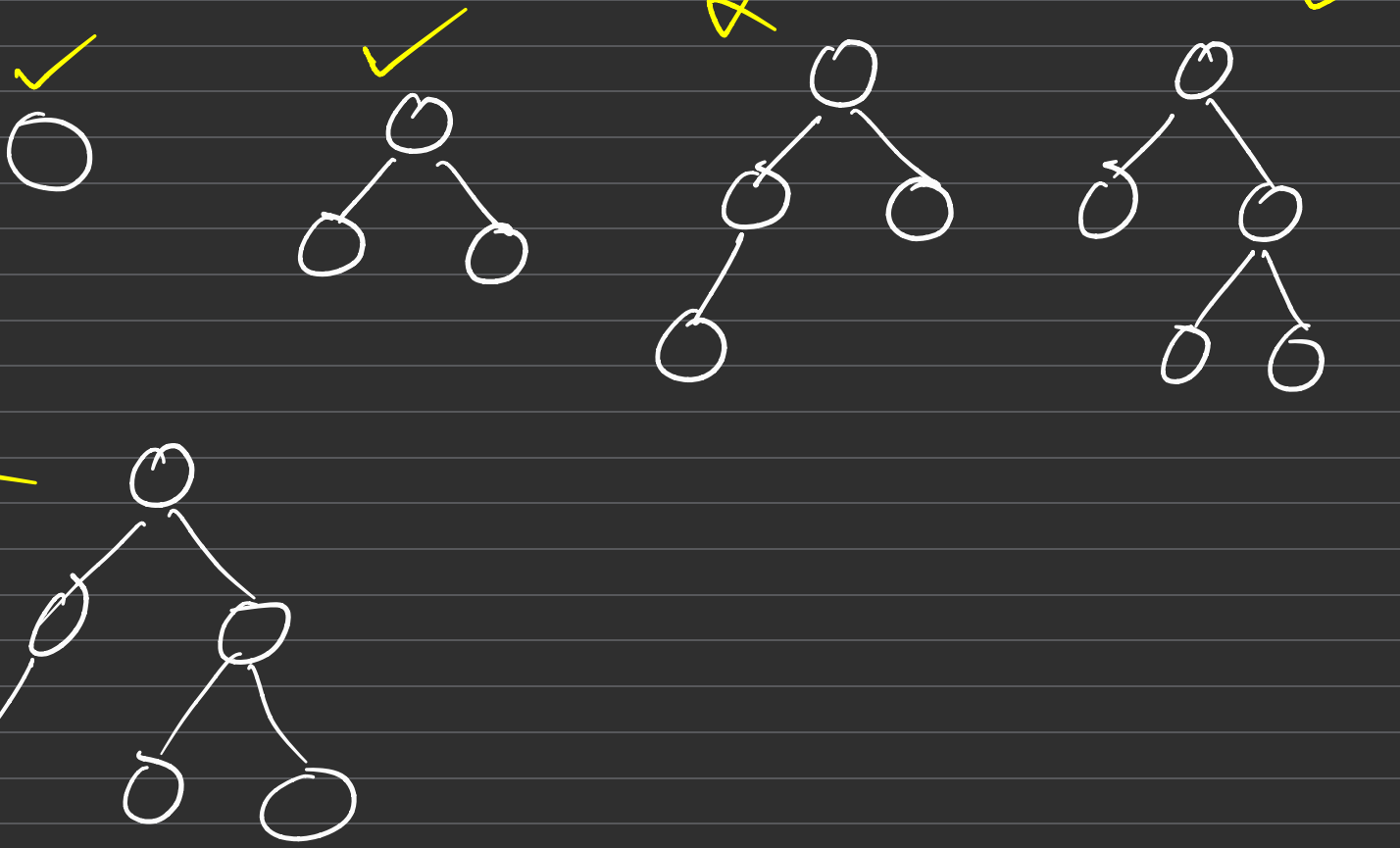




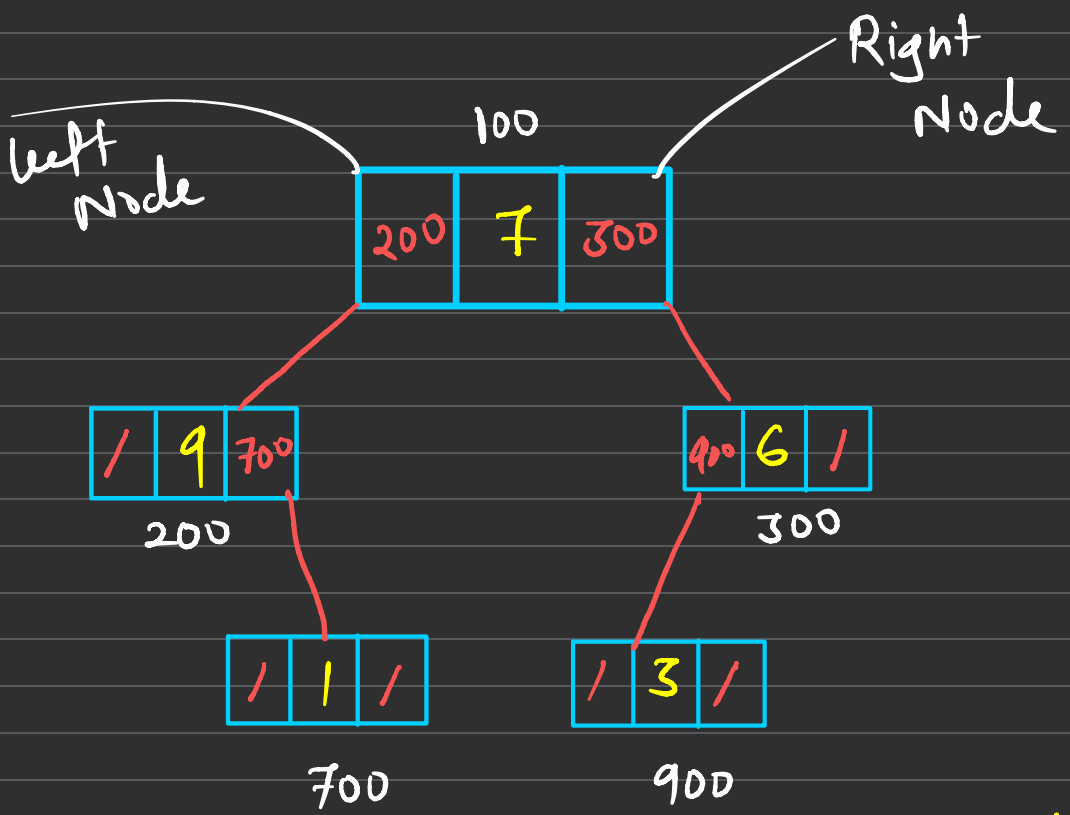




FBT



Structure of Tree




```
class Node
{
    int data
    Node * next
};
```


```
class Node
{
    int data
    Node * left
    Node * right
};
```

Tree Traversals Binary Tree

Pre-Order Traversal

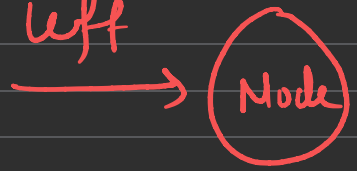
Post-Order Traversal

In-Order Traversal

Level-Order Traversal

_ Pre-Order Traversal

print
Root Left Right

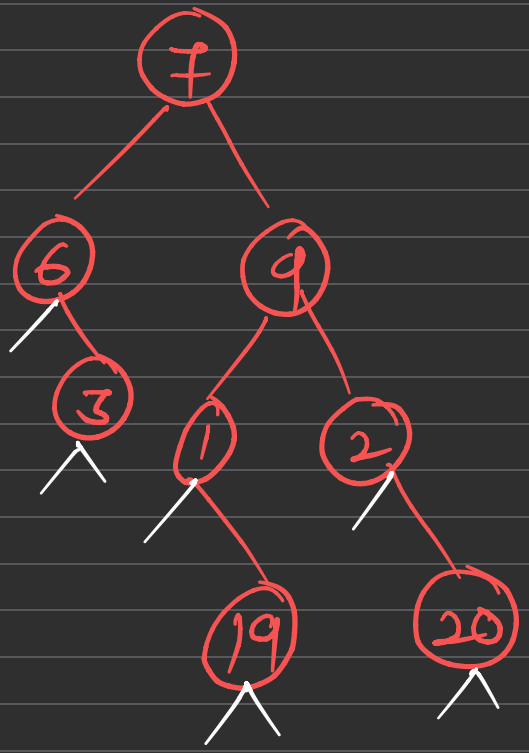


1 2 4 5 6 7 3

In-Order Traversal

Left **Root** Right
(print)

4 2 6 5 7 1 3



Pre-Order =

7 6 3 9 1 19 20

Inorder =

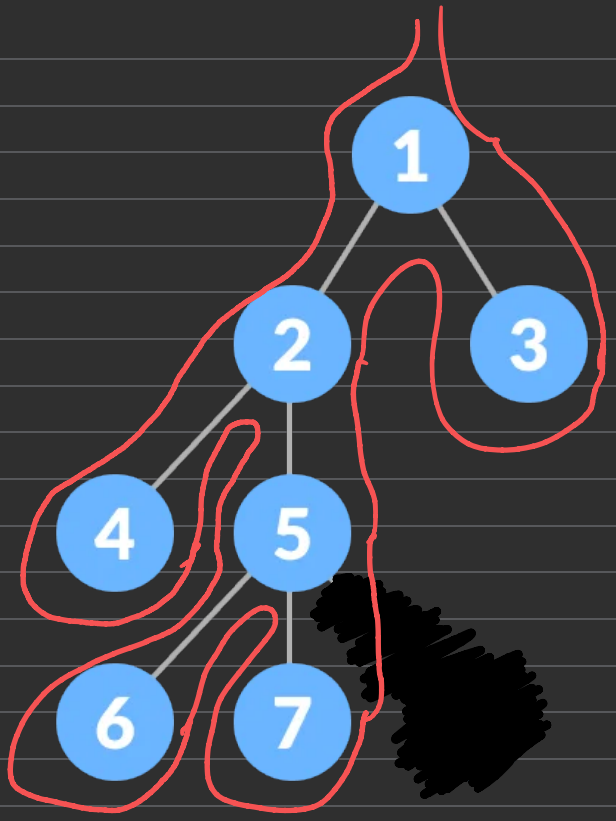
6 3 7 1 19 9 2 20

post-Order =

3 6 19 1 20 2 9 7

Post-Order Traversal

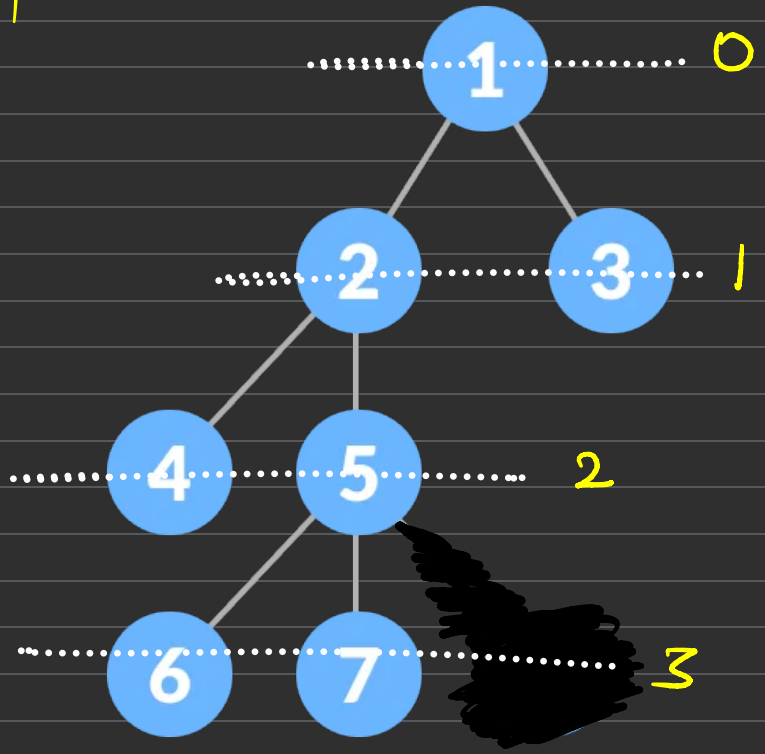
Left Right Root
print



4 6 7 5 2 3 1

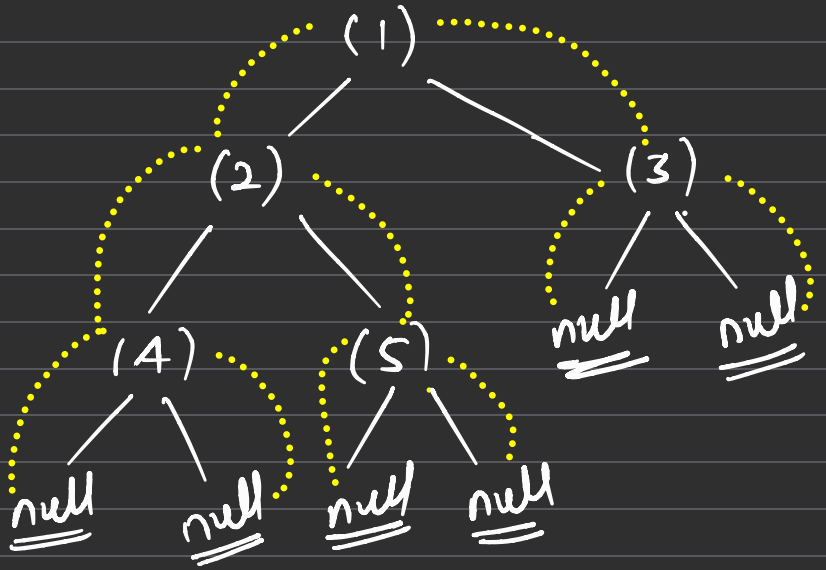
Level-Order Traversal

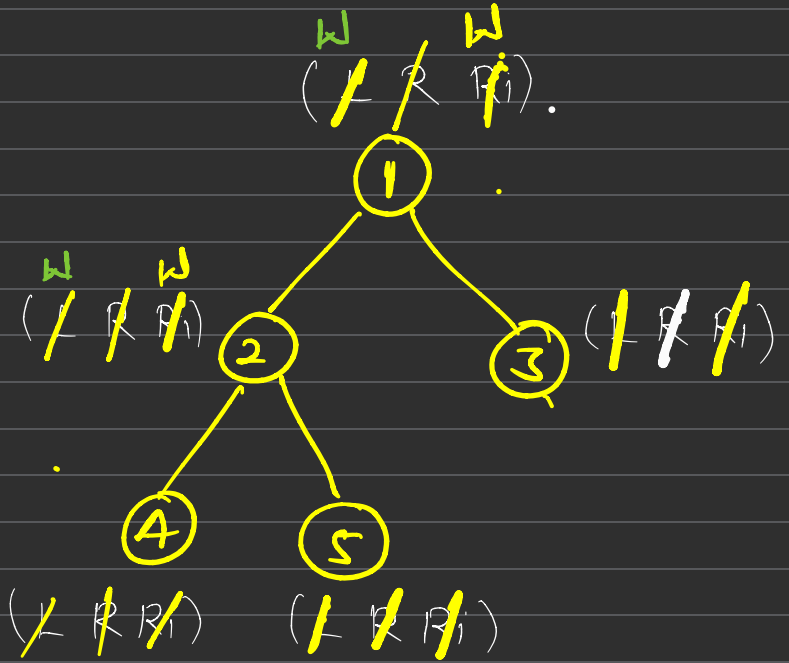
1 2 3 4 5 6 7



- In-order Traversal

(Left Root Right)



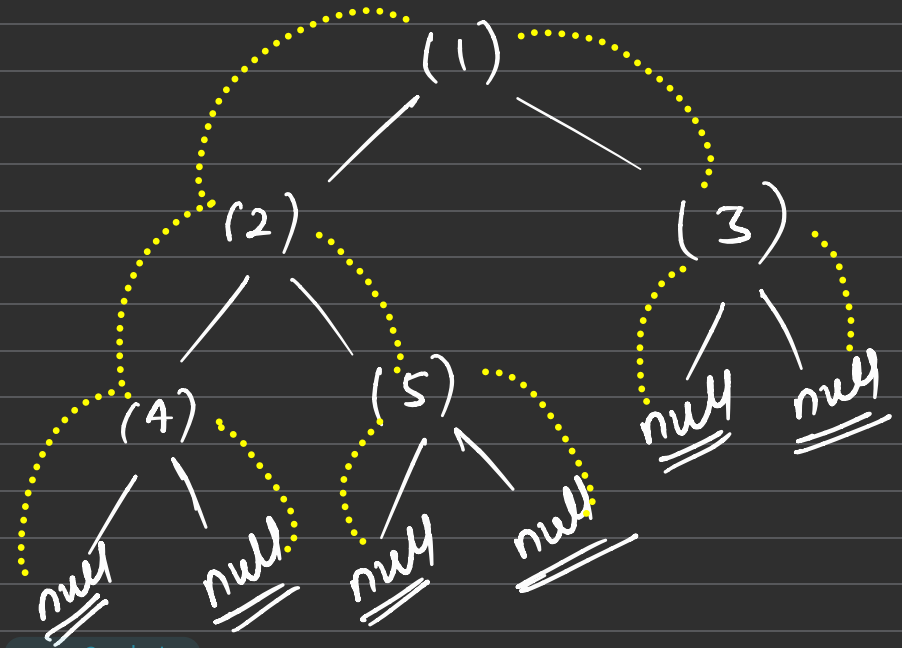


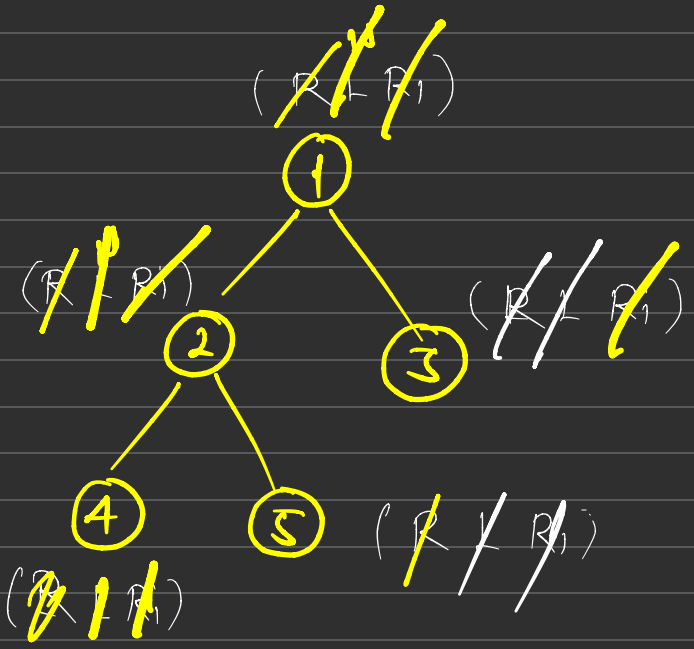
4 2 5 1 3

```
void inorder(TreeNode *root, vector<int> &ans){
    if (root == NULL) return;

    // left root(print) right
    inorder(root -> left, ans);

    ans.push_back(root -> val); // print - root

    inorder(root -> right, ans);

}
```

Pre-Order Traversal

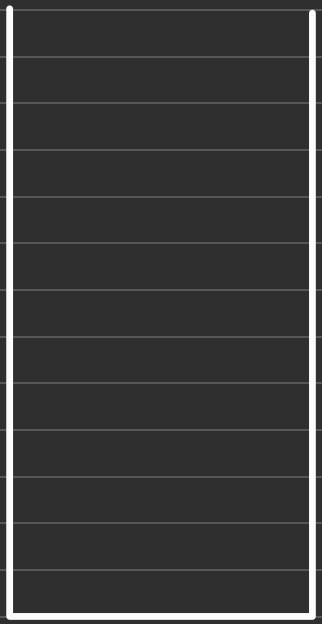
Root Left Right





1 2 4 5 3

Level-order Traversal



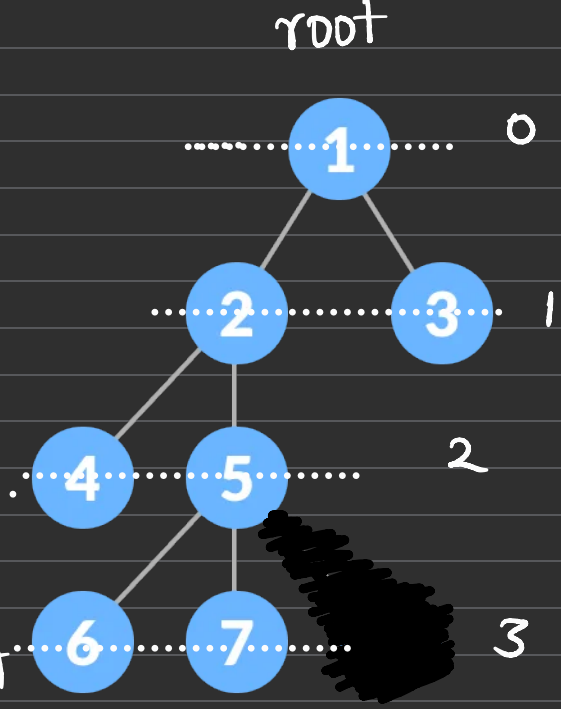
Queue
<Node*>

Node * node = q.front()
q.pop()

node = [7]

node->left

node->right



1 2 3 4 5 6 7

```
queue < Node * > q.
q.push (root)

while (!q.empty())
{   Node * node = q.front()
    q.pop()
    print (node → data)

    if (node → left != null)
        q.push (node → left)
    if (node → right != null)
        q.push (node → right)
}
```

node

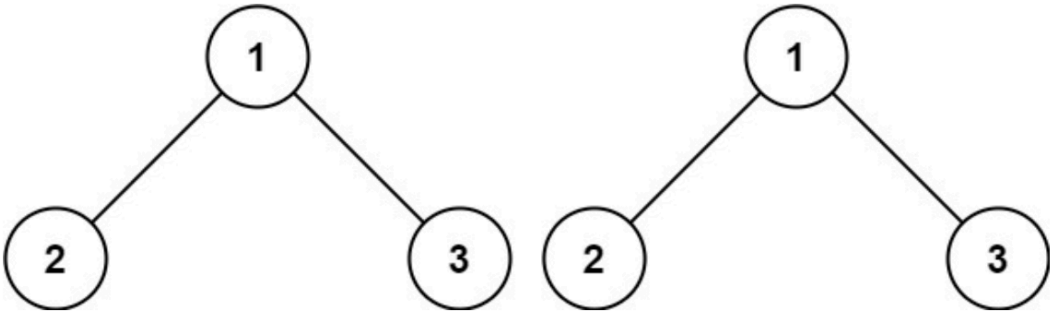
100. Same Tree

Easy | Topics | Companies | Re-do

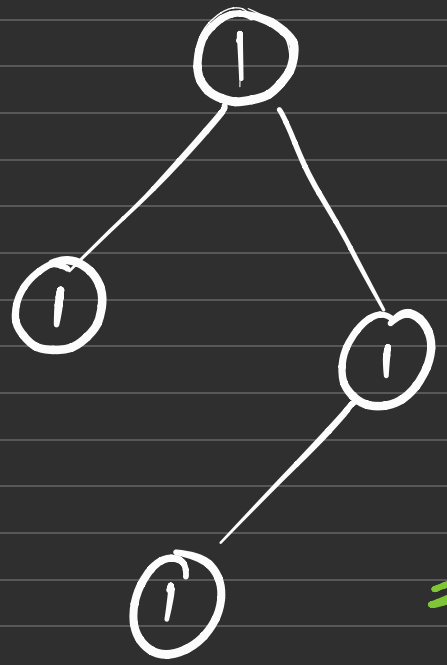
Given the roots of two binary trees `p` and `q`, write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1: (1 2 3) (1 2 3)

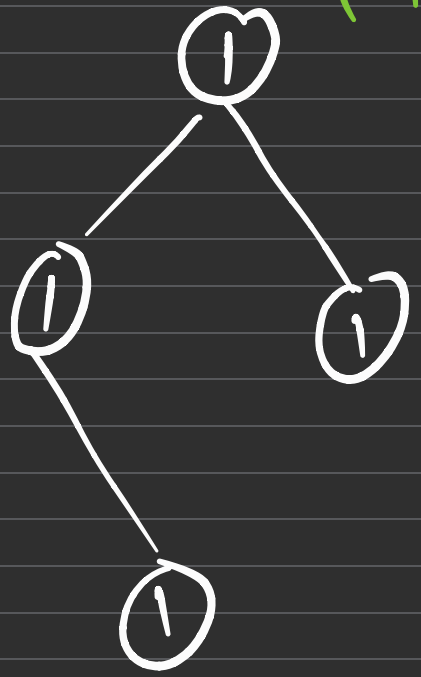


(1 1 1 1)



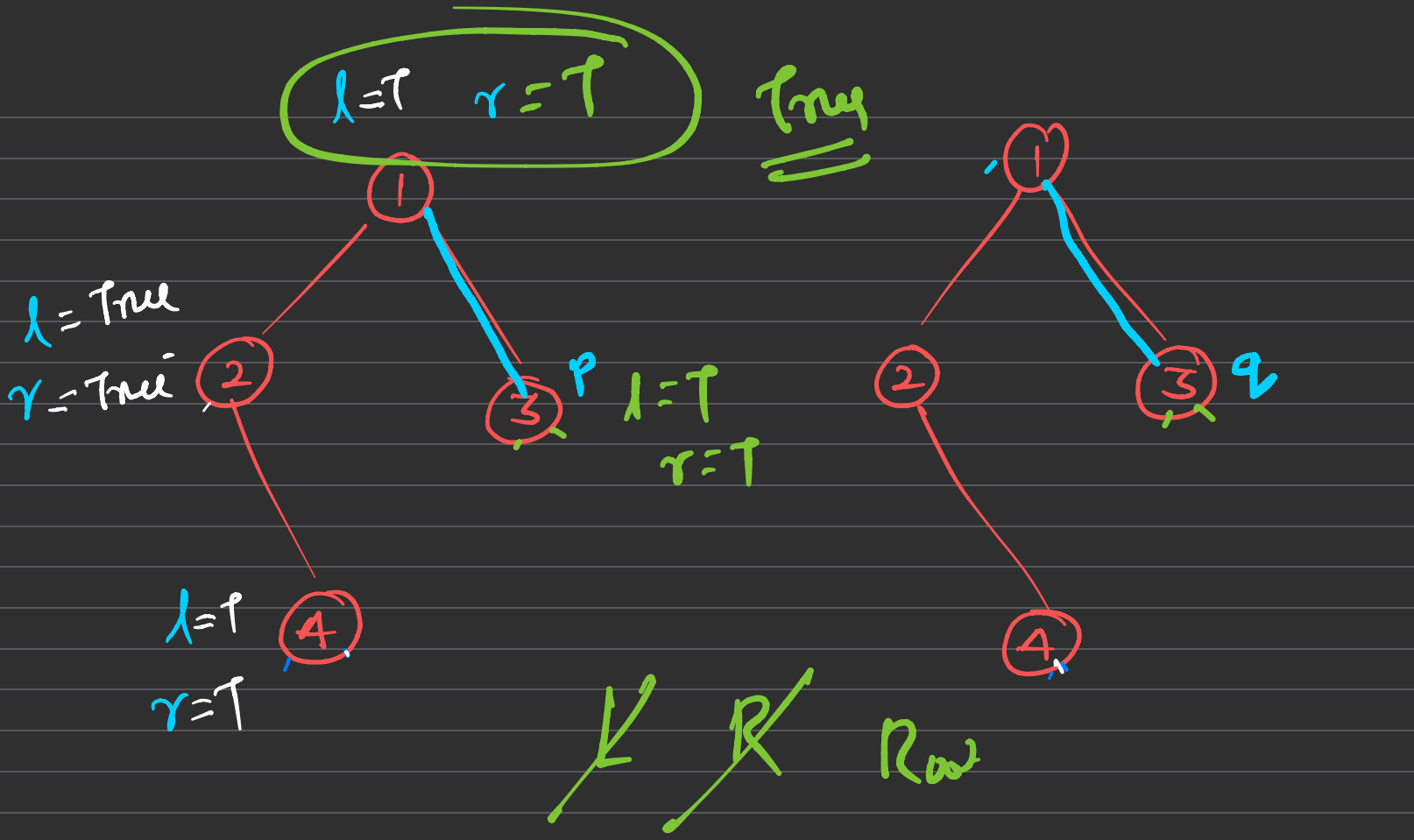
False

(1 1 1 1)



True ✅

p	q	isEqual
Null	Null	Yes
Null	Not Null	No
Not Null	Null	No
Value 9	10	No

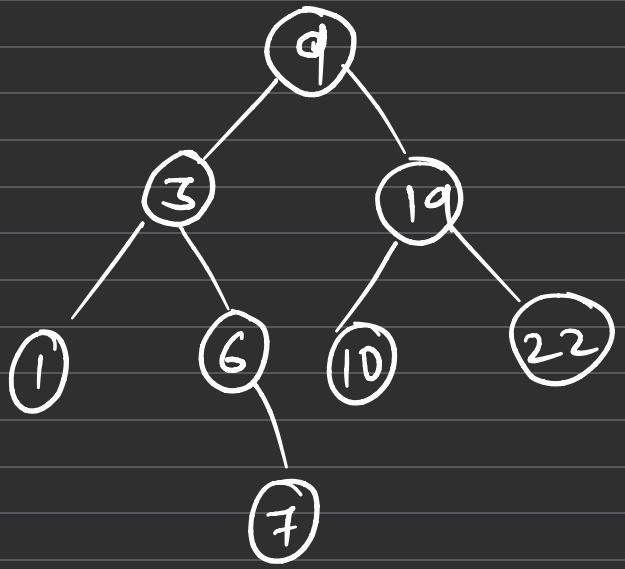


Binary Search Tree

- Node which have smaller value than Root is on left side

- Node which have greater value than Root is on Right side.

- Inorder Traversal always give data in increasing.



Inorder =

1 3 6 7 9 10 19 22

701. Insert into a Binary Search Tree

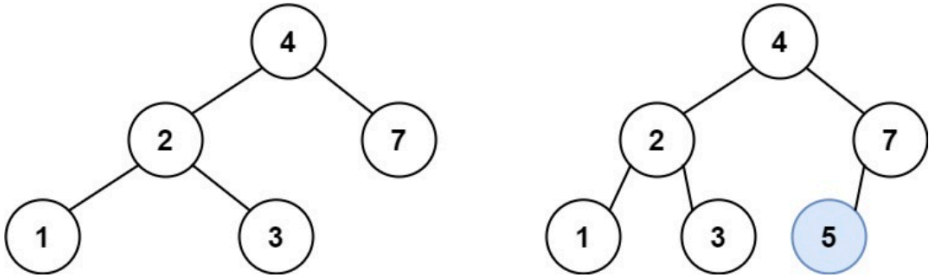
Solved

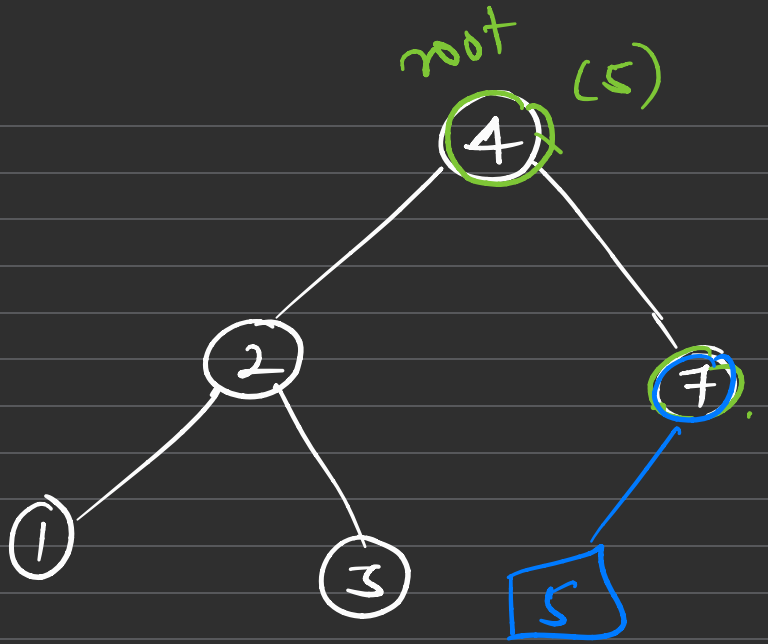
Medium | Topics | Companies | Re-do

You are given the `root` node of a binary search tree (BST) and a `value` to insert into the tree. Return *the root node of the BST after the insertion*. It is **guaranteed** that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return **any of them**.

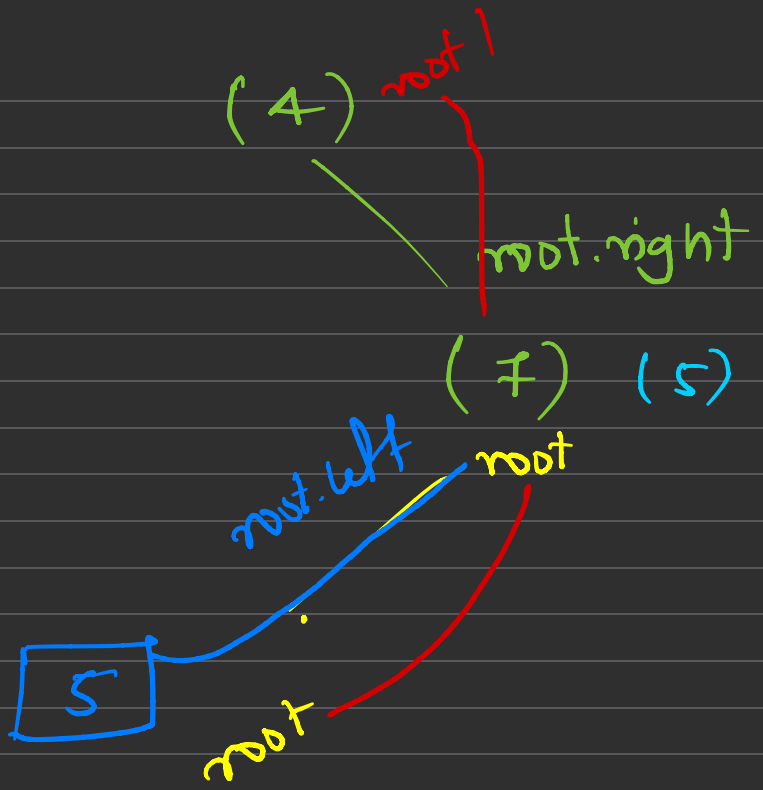
Example 1:





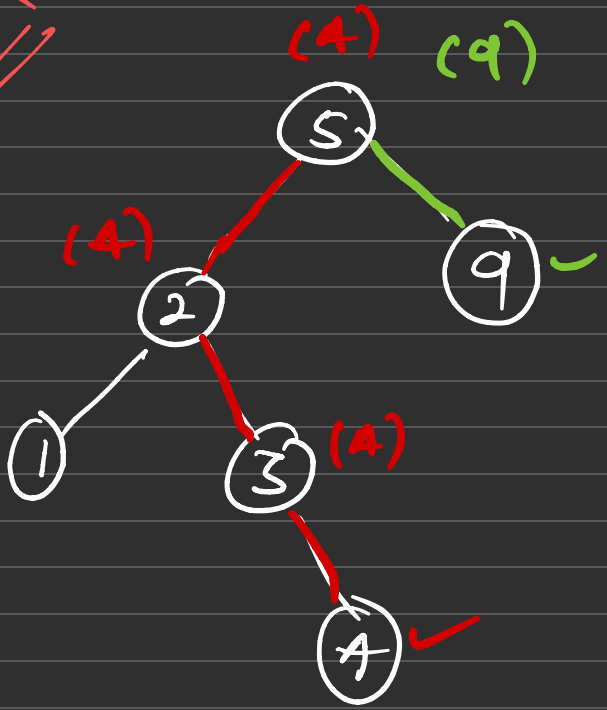
value = 5

```
Node * BST (root, data)
{
    if (root == null)
    {
        Node * node = new Node(data)
        return node
    }
    if (data > root.val)
        root -> right = BST (root.right)
    else
        root -> left = BST (root.left)

    return root
}
```



AVL Tree (Balanced)

BST



(4)

Snehal = 3

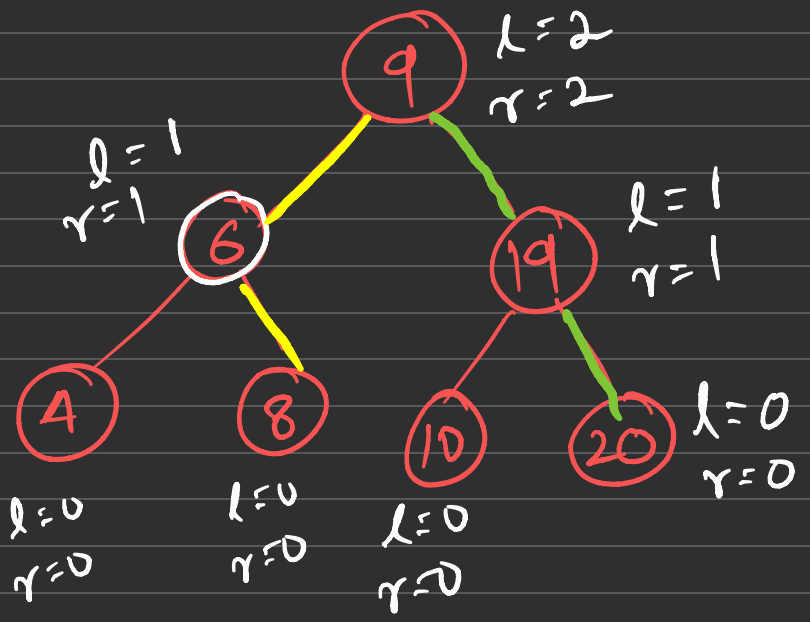
Harsh = 9 = 1

* Balanced Tree:

Each Node should have Balance fact (0, 1, -1)

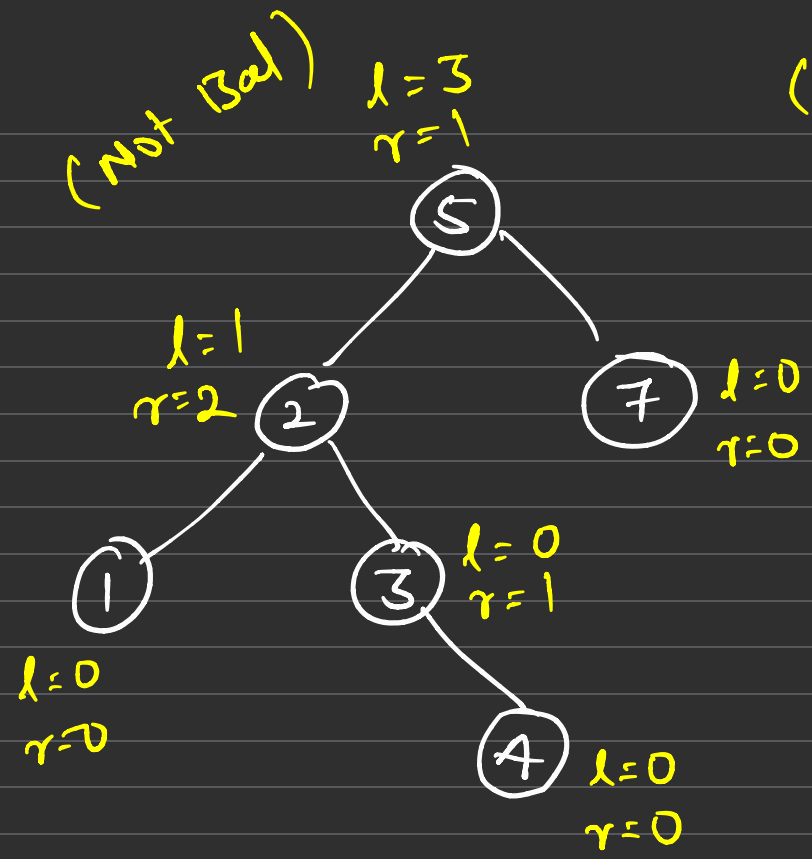
Balance factor = longest path on left − longest path on Right

(0, 1, -1)

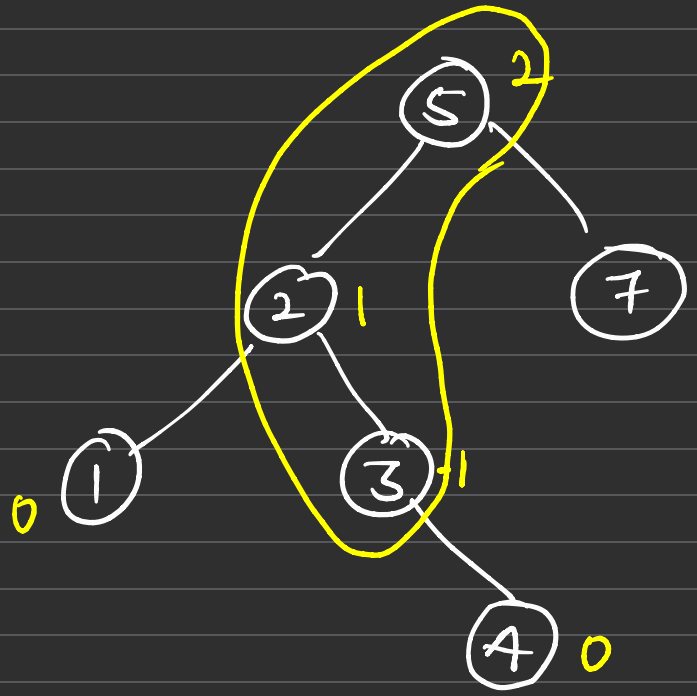


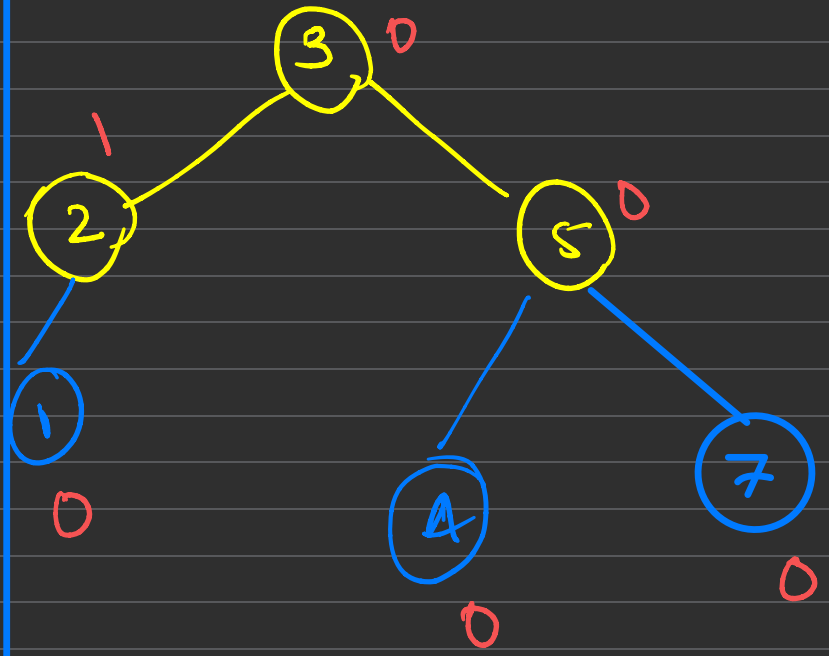
Shehal = 2
(20)

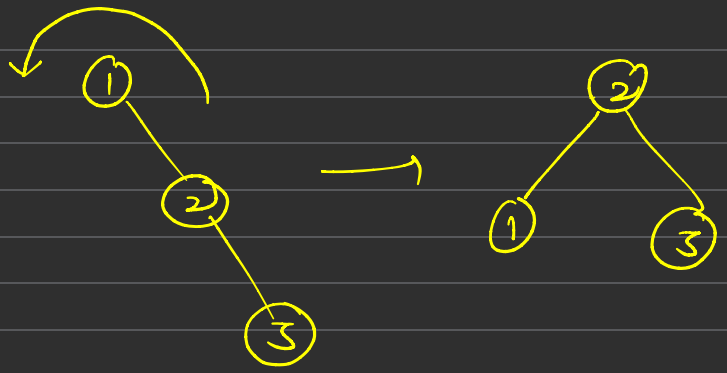
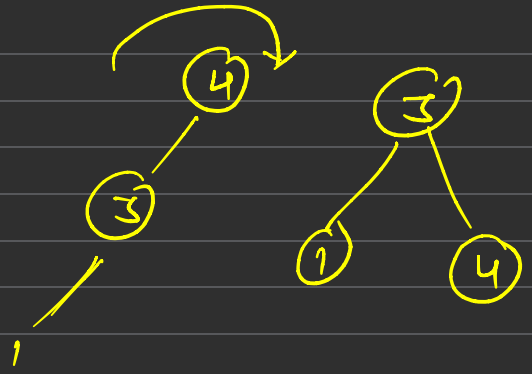
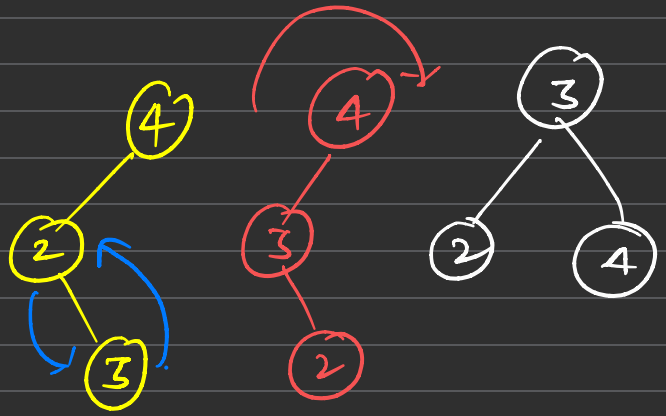
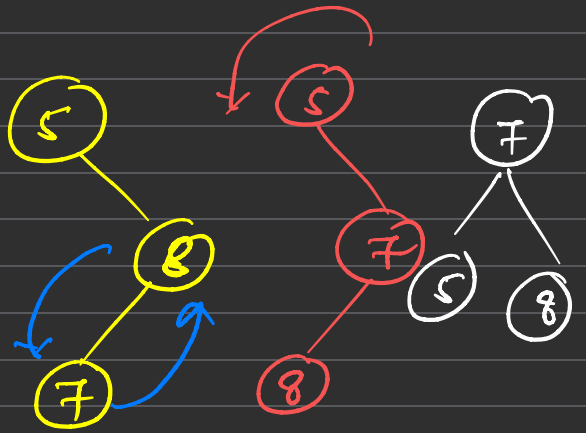
Harsh = 2
(8)



(0, 1, -1)





700. Search in a Binary Search Tree

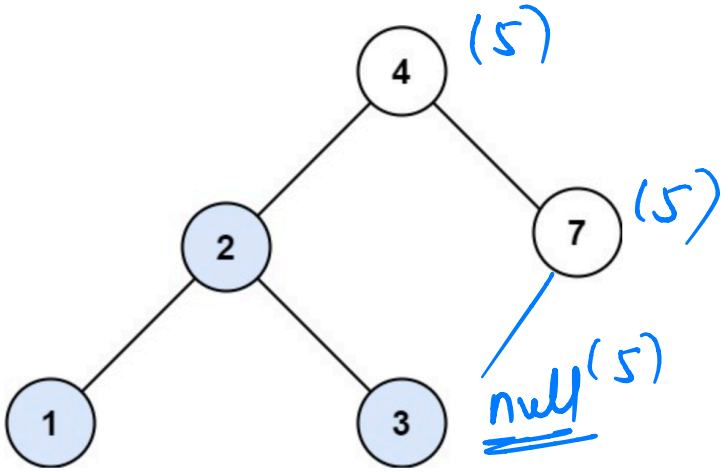
Solved

Easy | Topics | Companies | Re-do

You are given the `root` of a binary search tree (BST) and an integer `val`.

Find the node in the BST that the node's value equals `val` and return the subtree rooted with that node. If such a node does not exist, return `null`.

Example 1:




```
Input: root = [4,2,7,1,3], val = 2
Output: [2,1,3]
```

```
Node * Search ( root , val )
{   if ( root == NULL)
        return  NULL

    if (root.val == val)
        return root

    if (root.val < val)
        return Search (root.right, val)
    else
      return Search ( root.left, val
}
```

root val
a = b
a < b
a > b

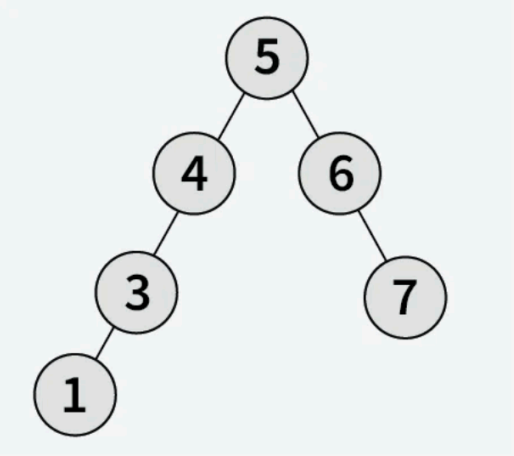
Minimum element in BST

Difficulty: **Basic** Accuracy: **70.95%** Submissions: **218K+** Points: **1** Average Time: **15m**

Given the root of a **Binary Search Tree**. The task is to find the minimum valued element in this given BST.

Examples

Input: root = [5, 4, 6, 3, N, N, 7, 1]



Output: 1

```
void minBST (root, ans)
{
    if (root == null)
        return;

    ans = root.val

    minBST (root.left, ans)
}

main()
    ans = INT_MAX
    minBST (root, ans)
```

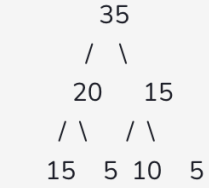
Children Sum in a Binary Tree

Difficulty: **Medium** Accuracy: **51.58%** Submissions: **206K+** Points: **4** Average Time: **20m**

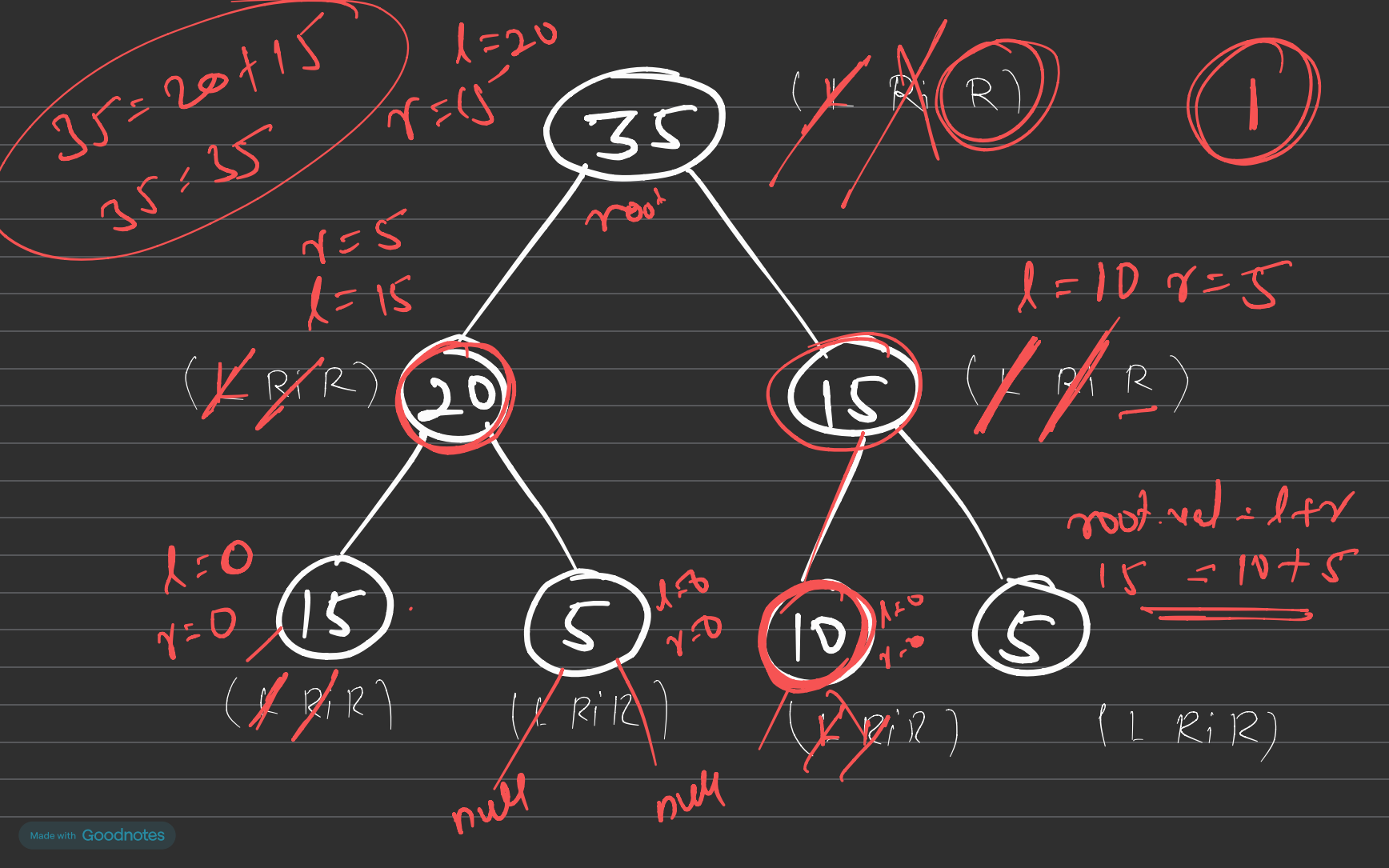
Given a binary tree having **n** nodes. Check whether all of its nodes have a value equal to the sum of their child nodes. Return 1 if all the nodes in the tree satisfy the given properties, else it returns 0. For every node, the data value must be equal to the sum of the data values in the left and right children. Consider the data value 0 for a NULL child. Also, leaves are considered to follow the property.

Examples:

Input:
Binary tree



Output: 1
Explanation:
Here, every node is sum of its left and right child.


35 = 20 + 15
35 = 35
l = 20
r = 15
35
root
(L Ri R)
1
r = 5
l = 15
20
(L Ri R)
15
l = 10 r = 5
(L Ri R)
root.val = l + r
15 = 10 + 5
l = 0
r = 0
15
(L Ri R)
5
l = 0
r = 0
(L Ri R)
10
l = 0
r = 0
(L Ri R)
5
(L Ri R)
null
null
Made with Goodnotes

Kth largest element in BST

Difficulty: **Easy** Accuracy: **49.31%** Submissions: **172K+** Points: **2**

Given a **Binary Search Tree**. Your task is to complete the function which will return the **kth largest** element without doing any modification in the Binary Search Tree.

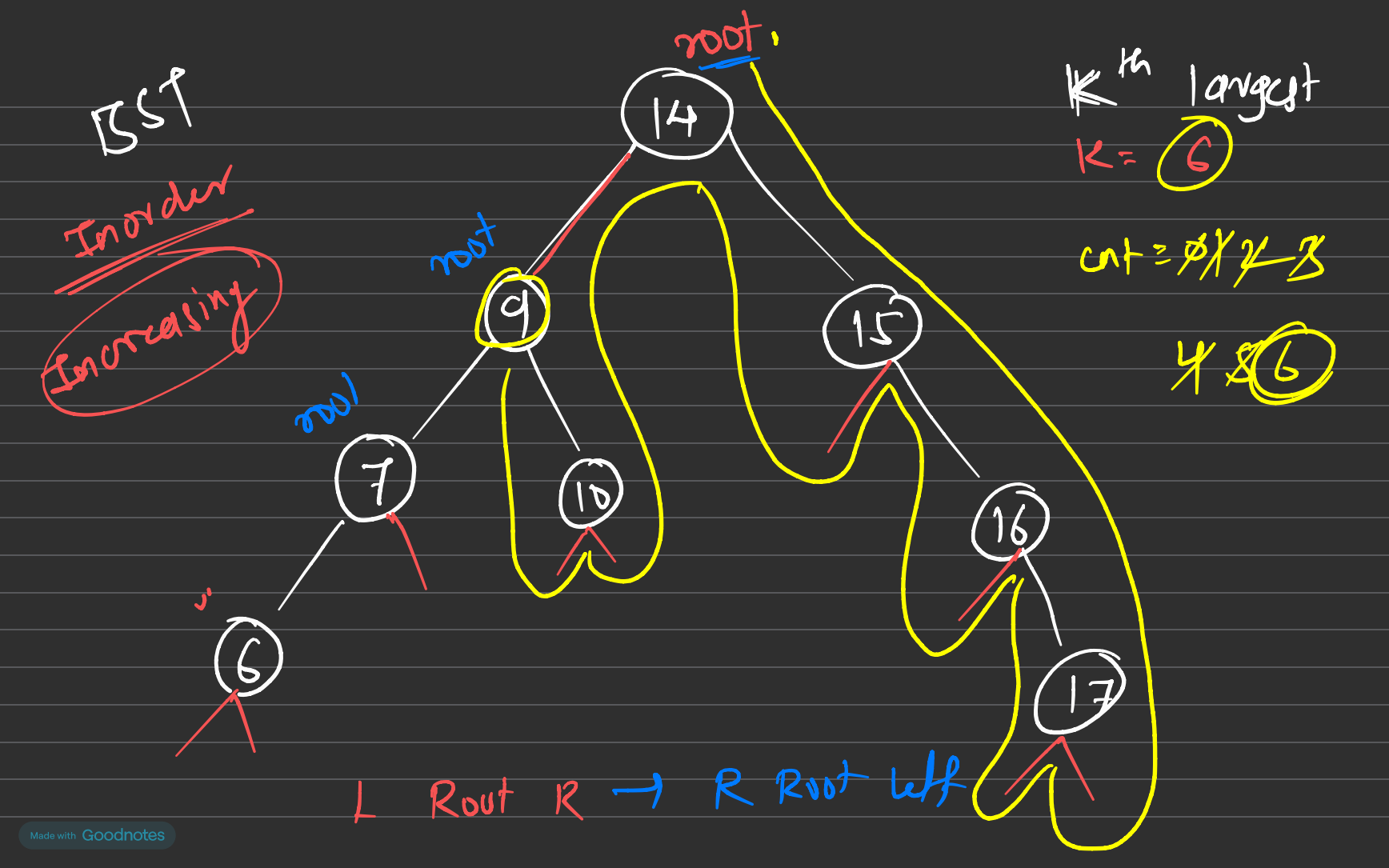
Examples:

Input:

k = 2
Output: 4
Explanation: 2nd Largest element in BST is 4

Input:

k = 1
Output: 10
Explanation: 1st Largest element in BST is 10


BST
Inorder
Increasing
root
14
9
15
7
10
16
6
17
root
root
K^{th} largest
K = 6
cnt = 0 1 2 3
4 5 6
L Root R → R Root Left

6 7 9 10 14 15 16 17

2 Root N

K^th largest

3^rd largest